

Quel rôle pour `getByRole` ?

WyTaSoft · Mehdi TAJMOUATI

On part de la balise vue dans le DOM (F12), on trouve la ligne, on a le rôle.

`getByRole` ne lit pas la balise mais l'**arbre d'accessibilité**, ce qu'annonce un lecteur d'écran. Un attribut `role="..."` explicite l'emporte toujours sur le rôle implicite de la balise.

Actions et navigation

HTML	RÔLE	EXEMPLE
<code><button>Ajouter</button></code>	<code>button</code>	<code>getByRole('button', { name: 'Ajouter' })</code>
<code><input type="submit" value="Envoyer"></code>	<code>button</code>	<code>getByRole('button', { name: 'Envoyer' })</code>
<code><input type="button"></code> , <code>reset</code> , <code>image</code>	<code>button</code>	
<code>Panier</code>	<code>link</code>	<code>getByRole('link', { name: 'Panier' })</code>
<code><a></code> sans <code>href</code>	<i>aucun</i>	⚠ introuvable par rôle

Titres et texte

HTML	RÔLE	EXEMPLE
<code><h1></code> ... <code><h6></code>	<code>heading</code>	<code>getByRole('heading', { name: 'Vos commandes', level: 1 })</code>
<code><p></code>	<code>paragraph</code>	préférer <code>getByText('...')</code>
<code><p role="alert"></code>	<code>alert</code>	<code>getByRole('alert')</code>
<code><p role="status"></code> , <code><output></code>	<code>status</code>	<code>getByRole('status')</code>
<code></code>	<code>img</code>	<code>getByRole('img', { name: 'Logo' })</code>
<code></code>	<i>aucun</i>	⚠ décorative, introuvable : c'est voulu
<code><hr></code>	<code>separator</code>	
<code><div></code> , <code></code>	<code>generic</code>	⚠ ne pas cibler par rôle : <code>getByText</code> ou <code>getById</code>

Champs de formulaire

HTML	RÔLE	EXEMPLE
<code><input type="text"></code> , email, tel, url, sans type	textbox	<code>getByRole('textbox', { name: 'Adresse email' })</code>
<code><input type="password"></code>	textbox	<code>getByRole('textbox', { name: 'Mot de passe' })</code> ou plus court <code>getByLabel('Mot de passe')</code>
<code><textarea></code>	textbox	
<code><input type="search"></code>	searchbox	<code>getByRole('searchbox', { name: 'Rechercher' })</code>
<code><input type="number"></code>	spinbutton	<code>getByRole('spinbutton', { name: 'Quantité' })</code>
<code><input type="checkbox"></code>	checkbox	<code>getByRole('checkbox', { name: 'Se souvenir de moi' })</code>
<code><input type="radio"></code>	radio	<code>getByRole('radio', { name: 'Livraison express' })</code>
<code><input type="range"></code>	slider	
<code><select></code> simple	combobox	<code>getByRole('combobox', { name: 'Catégorie' })</code>
<code><select multiple></code> ou <code>size > 1</code>	listbox	
<code><option></code>	option	<code>getByRole('option', { name: 'Périphériques' })</code>
<code><input list="..."></code> avec <code><datalist></code>	combobox	
<code><fieldset></code> + <code><legend></code>	group	<code>getByRole('group', { name: 'Adresse' })</code>
<code><form></code> avec un nom (<code>aria-label ...</code>)	form	sans nom : <i>aucun</i>

Pour un champ, `getByLabel('...')` fait souvent le même travail en plus court.

Listes et tableaux

HTML	RÔLE	EXEMPLE
<code></code> , <code></code>	<code>list</code>	<code>getByRole('list')</code>
<code></code>	<code>listitem</code>	<code>getByRole('listitem').filter({ hasText: 'Clavier mécanique' })</code>
<code><table></code>	<code>table</code>	<code>getByRole('table')</code>
<code><tr></code>	<code>row</code>	<code>getByRole('row', { name: /Souris ergonomique/ })</code>
<code><td></code>	<code>cell</code>	<code>getByRole('cell', { name: '49,00 €' })</code>
<code><th></code> dans <code><thead></code>	<code>columnheader</code>	<code>getByRole('columnheader', { name: 'Montant' })</code>
<code><th scope="row"></code>	<code>rowheader</code>	
<code><thead></code> , <code><tbody></code> , <code><tfoot></code>	<code>rowgroup</code>	

`getByRole('row')` compte aussi la ligne du `<thead>` : un panier de deux articles a **trois** row .

Structure de la page (repères)

HTML	RÔLE	REMARQUE
<code><header></code>	<code>banner</code>	seulement hors de <code><main></code> , <code><article></code> , <code><section></code> ...
<code><nav></code>	<code>navigation</code>	
<code><main></code>	<code>main</code>	
<code><footer></code>	<code>contentinfo</code>	même condition que <code><header></code>
<code><aside></code>	<code>complementary</code>	
<code><section></code> avec un nom	<code>region</code>	sans nom : <i>aucun</i>
<code><article></code>	<code>article</code>	
<code><dialog></code> , <code>role="dialog"</code>	<code>dialog</code>	<code>getByRole('dialog', { name: 'Cookies' })</code>
<code><progress></code>	<code>progressbar</code>	

Composants riches (`role="..."` explicite)

Les bibliothèques d'interface (Angular Material, MUI, PrimeNG...) posent un `role` sur des `<div>`. C'est ce rôle que lit `getByRole`.

COMPOSANT	RÔLES	EXEMPLE
Onglets	<code>tablist</code> <code>tab</code> <code>tabpanel</code>	<code>getByRole('tab', { name: 'Avis', selected: true })</code>
Menu déroulant	<code>menu</code> <code>menuitem</code> <code>menuitemcheckbox</code> <code>menuitemradio</code>	<code>getByRole('menuitem', { name: 'Copier' })</code>
Interrupteur on/off	<code>switch</code>	<code>getByRole('switch', { name: 'Wifi', checked: true })</code>
Bulle d'aide	<code>tooltip</code>	<code>getByRole('tooltip')</code>
Confirmation bloquante	<code>alertdialog</code>	<code>getByRole('alertdialog', { name: 'Supprimer ?' })</code>
Grille de données	<code>grid</code> <code>row</code> <code>gridcell</code>	<code>getByRole('gridcell', { name: '49,00 €' })</code>
Arborescence	<code>tree</code> <code>treeitem</code>	<code>getByRole('treeitem', { name: 'Documents' })</code>

HTML natif moins courant

HTML	RÔLE	REMARQUE
<code><input type="file"></code>	<code>button</code>	⚠️ surprenant ; pour envoyer un fichier : <code>setInputFiles</code>
<code><input type="date"></code> , <code>type="color"</code>	<code>textbox</code>	
<code><details></code> / <code><summary></code>	<code>group</code>	
<code><figure></code> + <code><figcaption></code>	<code>figure</code>	le nom vient de la légende
<code><dl></code> / <code><dt></code> / <code><dd></code>	<code>term</code> / <code>definition</code>	pratique sur les fiches produit
<code><meter></code>	<code>meter</code>	
<code><search></code>	<code>search</code>	
<code><svg role="img" aria-label="..."></code>	<code>img</code>	les icônes
<code><iframe></code>	<code>iframe</code>	⚠️ pour agir dedans : <code>frameLocator</code> , pas <code>getByRole</code>

D'où vient le `name` ?

C'est le **nom accessible**, pas l'attribut HTML `name`.

ÉLÉMENT	LE NOM VIENT DE...
bouton, lien, titre, cellule	son texte visible
<code><input type="submit"></code>	son attribut <code>value</code>
champ de formulaire	son <code><label></code> , <code>aria-label</code> ou <code>aria-labelledby</code>
image	son <code>alt</code>
n'importe quel élément	<code>aria-label</code> s'il est présent, qui remplace le texte
ligne de tableau (<code>row</code>)	le texte de toutes ses cellules, bout à bout

Par défaut, `name` est **partiel et insensible à la casse** : `{ name: 'Panier' }` trouve aussi « Ajouter au panier ».

```
page.getByRole('link', { name: 'Panier', exact: true })
page.getByRole('button', { name: /^Ajouter/ }) // ou une regex
```

Les autres options

```
getByRole('heading', { level: 2 }) // h2 seulement
getByRole('checkbox', { checked: true }) // case cochée
getByRole('checkbox', { checked: 'mixed' }) // partiellement cochée
getByRole('button', { disabled: true }) // bouton désactivé
getByRole('button', { expanded: true }) // aria-expanded
getByRole('button', { pressed: true }) // aria-pressed
getByRole('option', { selected: true }) // option choisie
getByRole('button', { includeHidden: true }) // éléments masqués
```

Les pièges

- **Lien sans `href`** : pas un lien pour l'accessibilité. C'est un défaut de l'application, à signaler.
- **Ligne d'en-tête** : `getByRole('row')` compte la ligne du `<thead>`.
- **Nom partiel** : une « strict mode violation » veut dire plusieurs éléments trouvés. Ajouter `exact: true` ou restreindre avec `.filter()`.
- **div cliquable** : un `<div onclick>` n'est pas un bouton. Si `getByRole('button')` ne le trouve pas, le test a raison : au clavier non plus, on ne pourra pas l'utiliser.
- `<header>`, `<footer>`, `<section>`, `<form>` : leur rôle dépend du contexte ou d'un nom. En cas de doute, vérifier dans l'arbre d'accessibilité.

Trouver le rôle sans deviner

1. **Codegen** : `npx playwright codegen http://localhost:3000` , puis « Pick locator » et survoler l'élément.
2. **DevTools** : F12 > *Elements* > panneau *Accessibility* : *Role* et *Name* calculés.
3. **Snapshot ARIA** : « Assert snapshot » dans le Codegen, ou dans un test :
`console.log(await page.locator('body').ariaSnapshot());` Chaque ligne donne **- rôle** **"nom"** , exactement ce qu'attend `getByRole` .

Ordre de préférence des locators

1. `getByRole` : ce que voit et utilise l'utilisateur
2. `getByLabel` : les champs de formulaire
3. `getByPlaceholder` , `getByText` , `getByAltText` , `getByTitle`
4. `getByTestId` : quand rien de visible ne suffit (badge, total)
5. CSS ou XPath : en dernier recours, jamais sur une classe générée